# Algospeak-Resilient Detection of Illicit Drug-Related Content Using Transformer-Based Machine Learning and a Browser Extension

Agalya M
*UG Scholar, Department of CSE (AI&ML),*
*KIT-Kalaignarkarunanidhi Institute of Technology,*
*Coimbatore, India*
*kit28.24bam003@gmail.com*

Gowri Shankar K
*UG Scholar, Department of CSE (AI&ML),*
*KIT-Kalaignarkarunanidhi Institute of Technology,*
*Coimbatore, India*
*kit28.24bam022@gmail.com*

Sowndhar A
*UG Scholar, Department of CSE (AI&ML),*
*KIT-Kalaignarkarunanidhi Institute of Technology,*
*Coimbatore, India*
*kit28.bam303@gmail.com*

Rajalingam B*
*Assistant Professor, Department of CSE (AI&ML),*
*KIT-Kalaignarkarunanidhi Institute of Technology,*
*Coimbatore, India*
*brajalingamnec@gmail.com*

**Abstract —** Social networking and online communication services have opened new routes for promoting, discussing and coordinating illegal drug activity. Such content is hard to catch because users who want to evade automated moderation deliberately alter words, or rely on emojis, abbreviations, changed spellings and coded phrases, a practice known as algospeak. This paper proposes a multimodal framework that detects drug-related content written in such obfuscated language and intervenes directly inside the browser. The framework brings together natural language processing, image classification, semantic preprocessing and a Chrome browser extension. Text taken from online content first goes through a normalization stage that maps 41 domain-specific emojis, slang terms and intentionally distorted spellings onto standard forms, and the normalized text is then scored by a fine-tuned DistilBERT classifier. In parallel, any accompanying imagery is scored by a MobileNet-based convolutional neural network, with text inside images recovered via RapidOCR. An asymmetric scoring rule merges the outputs into a single risk value, which is passed to the browser extension; depending on calibrated thresholds, the extension displays warnings or applies visual masking with a one-click reveal option. Evaluation on an algospeak-augmented dataset of 5,500 samples (Fleiss' $\kappa = 0.88$) and a curated drug image dataset demonstrates that the full multimodal system reaches 94.20% accuracy and an F1-score of 0.94, while normalization yields a 12.30 percentage-point gain over unnormalized text. The system achieves an end-to-end response time of roughly 185 ms on standard consumer CPU hardware.

**Keywords —** Drug-Related Content Detection, Algospeak, DistilBERT, Multimodal Learning, Semantic Normalization, MobileNetV2, Optical Character Recognition, Browser Extension, Real-Time Detection

## I. INTRODUCTION

People now rely on online communication to share information and engage with communities, but public social networks, forums and similar platforms can also be exploited for illegal drug activity. They give misusers a wide audience, fast communication and the chance to publish content with very little effort, so automatically spotting suspicious material has become a key concern for online safety systems.Spotting such content is hard because users rarely write in plainly drug-related terms. They may distort words, insert spaces or symbols, swap characters, shorten terms or express meaning through emojis, and they may coin informal expressions that members of a given online community understand but standard keyword filters miss. This kind of altered communication is generally called algospeak.

Keyword-based filtering is weak because it only looks for words and phrases already on its list, so changing the spelling of a term can slip past it. Characters can be replaced by symbols, letters can be spaced apart, or an emoji can stand in for the word. A human reader still grasps the meaning, although the surface form no longer matches the vocabulary stored in the filter.

Machine learning offers a way past literal keyword matching. Transformer-based language models weigh the relationships among the words of a sentence and therefore analyze context more effectively. Text alone, however, is often insufficient evidence, since posts may include pictures of substances, packaging or other visual material while the caption stays brief or intentionally vague.

This work accordingly merges text and image analysis. For text, a semantic normalization stage feeds a fine-tuned DistilBERT classifier; for images, a MobileNet-based convolutional neural network is used. The two outputs are merged into a single risk score, and a browser extension on the client side acts on that score.

A further aim is to shorten the gap between content appearing in a browser and the user receiving an intervention. Rather than depending solely on analysis after publication, the architecture lets the browser extension watch page content, send the relevant data to the backend, get a prediction back, and alter what is displayed whenever the predicted risk passes the configured threshold.

The major contributions described in this work are:

- A dedicated pre-transformer normalization stage with 41 hand-crafted rules (19 DEA-aligned drug emojis and 22 leetspeak patterns) that maps obfuscated terms back to canonical words before tokenization, preventing subword token fragmentation.
- A dual-stream multimodal pipeline combining DistilBERT for contextual text classification, MobileNetV2 for visual classification, and RapidOCR for text-in-image extraction, supported by a sub-millisecond TF-IDF fallback.
- An asymmetric cross-modal risk fusion scoring rule that prevents visual evidence from being diluted when an image contains no text, while mutually reinforcing risk

when both visual contraband and promotional text overlays are present.

• A client-side intervention mechanism built on Manifest V3 utilizing MutationObserver DOM listeners, local WeakSet caching, and progressive visual masking (warning banners and CSS blur filters with one-click user reveal).

• An empirical benchmark on a verified 5,500-sample algospeak dataset (Fleiss' $\kappa = 0.88$) and 691 curated images, providing latency profiling on consumer CPU hardware, normalization ablations, and error analysis.

## II. RELATED WORK

Earlier work on detecting drug-related content online spans conventional text classification, transformer-based language models, studies of informal terminology, multimodal learning and browser-level content intervention. Reports that local law enforcement still finds digital evidence in cybercrime cases difficult to handle [15] further show why automated screening of online content is a useful first line of defense.

### *A. Transformer-Based Detection*

Compared with earlier methods built mainly on keyword matching or bag-of-words features, transformer architectures have greatly improved contextual language processing. Narcotrace, for instance, uses BERT-based classification on drug-related social media content [2], and such models draw on context to separate the different senses of words and expressions. Alfatmi et al. additionally studied LLM-based detection of illicit messages on social networks [1].

Other studies enrich language models with domain knowledge; Hu et al. put forward a knowledge-prompted ChatGPT method for recognizing drug-related content [7]. Together these results show that contextual understanding helps when explicit keywords are not enough.

BERT-family and other deep learning models also serve different text understanding tasks. Narmadha et al. applied BERT to detect and correct bias in text [16], and further studies cover deep emotion analysis of consumer feedback [17] and a survey of data-driven customer sentiment analysis [18]. These works concentrate on interpreting ordinary text, not text that has been intentionally disguised, which is the case this paper targets.

Many existing approaches, though, work mainly as backend analysis systems, so results may only appear after content has been posted or processed by a platform. They therefore do not necessarily offer a way to intervene immediately in an individual user's browsing environment.

### *B. Algospeak and Informal Drug Terminology*

Modified language poses a separate problem for automated moderation. Users may rely on emojis, abbreviated words, phonetic spellings, unusual character sequences or community-specific vocabulary. The DEA has published a guide to emojis used as drug codes [11], and the influence of social media on how everyday language evolves [19] helps explain why this vocabulary shifts so quickly.

Song et al. presented JEDIS, which identifies drug-related jargon through delexicalized distant supervision [3], [4]; their work shows how contextual representations can help recognize terms that do not look like standard drug vocabulary. Other studies have explored context-aware transformer methods for informal or partly encrypted communication [6], and Gautam et al. looked at NLP and data analysis approaches for investigating drug trafficking over encrypted messengers [14]. Together they suggest that context matters when interpreting altered expressions.

Combining symbolic knowledge with neural learning has also been tried in cyber defense, as in the neuro-symbolic intrusion detection framework of Karthi et al. [20]. The present framework pairs the two in a similar way: a rule-based normalization stage prepares the input and a neural classifier makes the final decision.

Purely text-based approaches may also overlook the visual material attached to a post. For that reason, the proposed framework makes normalization an explicit preprocessing step, so that varied surface forms are converted into representations the later models can process.

### *C. Multimodal Detection*

Online content often contains both text and images, and a message with little text may come with an image that supplies more context. Multimodal learning methods tackle this by merging information from different input types, and work combining transformer-based text representations with CNN-based image representations has shown the value of using several modalities together [9]. Hu et al. likewise fused large-scale multimodal data to identify illicit drug dealers on Instagram [8].

Hayashi and Nojima studied image classification combined with real-time monitoring to find drug-related activity on social networking services [10], showing that visual analysis adds value when text alone is not enough.

Multimodal and multi-stream designs also feature in recent studies from our institution. Priya et al. proposed multi-modal contrastive learning with temporal consistency analysis to detect deepfake videos [21]; Bala et al. used dual-stream CNN–Transformer networks for layered cyber threat analytics [22]; Padmapriya et al. introduced adaptive fusion of CNN and Vision Transformer features for medical image classification [23]; and Vijayakumar and Ganesh presented a multimodal retrieval-augmented generation framework for context-aware semantic querying [24]. These designs merge information at the feature, contrastive or retrieval level, whereas the present system combines the output probabilities of its two branches through a weighted linear rule, which keeps inference quick enough for interactive browsing.

Text inside images is another source of evidence. Lightweight OCR pipelines, for example the LLM-assisted TamOCR framework for digitizing Tamil-Brahmi inscriptions [25], show that OCR can be efficient enough to feed language models, an idea reflected in the RapidOCR stage of our backend.

Even so, multimodal systems can be costly to run, and many implementations target offline or batch processing. This system instead stresses a lightweight inference pipeline that can work with a browser extension.

### *D. Browser-Based Intervention*

Once a browser extension has located relevant elements in the Document Object Model (DOM), it can alter the page content. Earlier work has used browser-side interventions to show warnings or change displayed information [12][13]. Work on adaptive user interface optimization with deep sequential learning [26] similarly shows that learned models can drive interface changes; here the learned risk score drives the interface action (warning, blur or block).

Through DOM observation, an extension can monitor dynamically generated page elements, which lets it catch content that appears after the initial load, such as posts or comments loaded by infinite scrolling.

Existing browser-based interventions, however, usually depend on rules written by hand, and such rules lose effectiveness when content uses contextual language, altered spellings or several modalities.

### *E. Research Gap*

The surveyed work reveals a split between advanced machine learning systems and browser-level intervention tools: transformer-based methods give contextual language analysis, multimodal systems merge text with visual information, and browser extensions offer a practical way to change what a user sees.

The proposed work brings these elements together in one pipeline, covering semantic normalization for altered language, text and image classification, weighted multimodal fusion and a browser-based intervention mechanism.

## III. SYSTEM ARCHITECTURE

### *A. Overall Architecture*

The proposed framework uses a client-server design in which the browser extension runs on the client side and a backend server performs the machine learning inference.

The architecture consists of three principal components:

- Browser Extension
- Machine Learning Backend
- Decision and Intervention Module

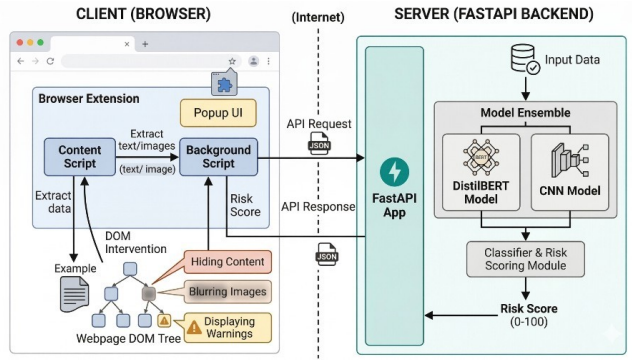



*Fig. 1. Overall system architecture showing the browser extension (client) and the FastAPI backend (server).*

In Fig (1), when a webpage is opened, the browser extension inspects relevant page elements and gathers visible text and image data, which is sent to the backend in an HTTP request.

The backend normalizes the text and runs the text and image classification stages, merges the predictions into one risk score and returns it to the extension, which decides whether the content stays as it is, gets a warning or is visually blocked.

### *B. Browser Extension*

The client side is a Chrome extension built on the Manifest V3 architecture.

A MutationObserver watches for changes in the page DOM. This is especially useful on modern sites that insert content after the first load, because it lets the extension react to newly added posts, comments or other relevant elements.

The extension performs the following main operations:

- identifies visible textual content;
- detects relevant image elements;
- removes unnecessary webpage markup during extraction;
- sends extracted information to the backend through HTTPS requests;
- receives the classification response;
- applies warnings or blur effects when necessary.

Because of this client-side design, detection runs while the user browses and no separate application interface is needed.

### *C. Machine Learning Backend*

The backend is written in Python with FastAPI and exposes an API through which the extension submits content for analysis.

Models are loaded once at server start-up rather than for every request, which lowers inference overhead.

The backend processing pipeline consists of four major stages:

- Normalize the submitted text.
- Classify the normalized text using DistilBERT.
- Analyze the supplied image using MobileNet.
- Combine the two prediction scores.

The backend responds with structured data holding the predicted risk score and the matching action.

### *D. Semantic Normalization*

Semantic normalization narrows the gap between ordinary expressions and their deliberately altered forms through three main transformations.

#### *1. Emoji Conversion*

A domain-specific dictionary maps 19 DEA-documented drug emojis onto standard text: controlled substance codes (❄ → 'cocaine', ~~an~~ → 'marijuana', → 'pills', → 'mushrooms', → 'heroin', → 'ice', → 'candy') and transaction slang markers ( → 'plug',

→ 'package', 💰 → 'cash', ⚡ → 'fast', 🔥 → 'fire', 💨 → 'smoke', 🎱 → 'eightball', ⛰️ → 'mountain', 🧪 → 'lab', 🔬 → 'tested', 💯 → 'hundred') [11].

### *2. Slang and Leetspeak Processing*

A rule set of 22 common orthographic substitutions maps obfuscated spellings onto standard vocabulary (e.g., dr*gs, d r u g s → 'drugs'; x*nax, x@nax → 'xanax'; c*caine, c0ca1ne → 'cocaine'; h*roin, h.e.r.o.i.n → 'heroin'; f*ntanyl, f3nt@nyl → 'fentanyl'; m3th → 'meth'; p1lls → 'pills'; w33d → 'weed'; 0xy → 'oxy').

### *3. Character Normalization*

Altered spellings, irregular spacing, and delimiter noise are cleaned so that symbol substitutions or separated characters are turned into a uniform representation prior to subword tokenization.

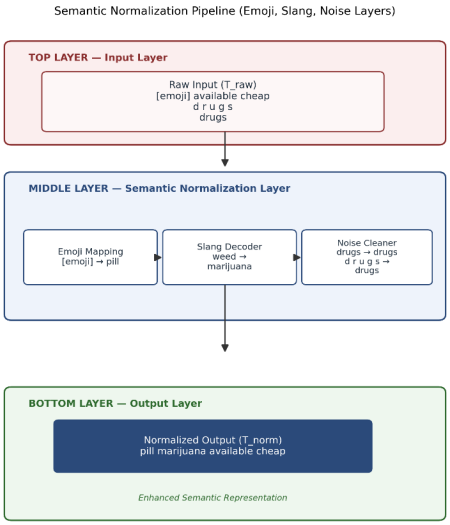



*Fig. 2. Three-layer semantic normalization pipeline: input, normalization (emoji, slang, noise), and output.*

In Fig (2), the normalized text is then supplied to the DistilBERT classifier.

### *E. Decision and Intervention*

The text and image model outputs are merged into one score, and the paper applies the following threshold scheme:

Score < 0.15 → Safe (No intervention, rendered normally)

0.15 ≤ Score < 0.35 → Warning (Light blur overlay with click-to-reveal)

Score ≥ 0.35 → Block (Heavy 16px blur overlay and keyword indicators)

The thresholds were chosen through validation-set analysis, balancing detection performance against false positives, with conservative bounds ensuring proactive shielding.

### *F. Workflow*

The complete processing sequence can be summarized as:

- The extension observes webpage content.
- Text and image information are extracted.
- The extracted information is sent to the backend.
- Text is normalized.
- DistilBERT processes the normalized text.
- MobileNet analyzes the image.
- The two outputs are combined.
- The resulting score is compared against the intervention thresholds.
- The browser applies the corresponding action.

## IV. METHODOLOGY

### *A. Input Collection*

The framework takes information extracted from webpages. Text may include ordinary sentences, informal expressions, emojis, abbreviations and deliberately altered spellings, while images may carry visual content tied to drug-related material.

Since these inputs vary considerably, preprocessing is carried out before classification.

### *B. Semantic Normalization*

Let the original input be represented by T_raw. The normalized text can be expressed as:

$$T_norm = f_norm\ (\ T_raw\ )$$

where f_norm represents the combined normalization operations.

Normalization covers emoji mapping, slang conversion, leetspeak handling and repair of irregular character spacing.

The aim is not to replace the language model but to give it a more uniform representation of content that would otherwise appear in many surface forms.

### *C. Text Classification with DistilBERT*

The normalized text is then fed to a fine-tuned DistilBERT model.

DistilBERT is a compact variant of BERT that cuts computational cost while keeping much of the original model's contextual language ability.

The classifier generates a probability value P_text between 0 and 1:

$$P_text \in [0,\ 1]$$

A larger value means the model is more confident that the input belongs to the drug-related class.

The reported setup has a binary classification head with sigmoid output.

### *D. Image Classification with MobileNet*

Images linked to page content are handled by a MobileNet-based convolutional neural network.

MobileNet relies on depthwise separable convolutions, which need less computation than standard convolutional architectures and therefore suit applications where inference speed matters.

Given an input image I, the model outputs the probability that the image belongs to the drug-related visual category.

### *E. Multimodal Fusion*

The system supports dual inference streams: direct text scanning and image/multimodal scanning. When evaluating images that may contain embedded text, screenshots, or promotional flyers, RapidOCR extracts textual content from the image region. If embedded text is present, both visual features and extracted text probabilities are fused according to:

**S = max(P_visual, 0.6 · P_visual + 0.4 · P_ocr)**

where P_visual is the MobileNetV2 probability and P_ocr is the DistilBERT probability on extracted image text.

For images without detectable text, the visual confidence score is retained directly: S = P_visual.

For text-only webpage elements, S = P_text.

This asymmetric formulation prevents false negatives when an illicit image contains no text, while boosting detection confidence when both visual contraband and promotional text are present.

Visual evidence thus directly contributes to the overall risk evaluation, as shown in Fig (3).

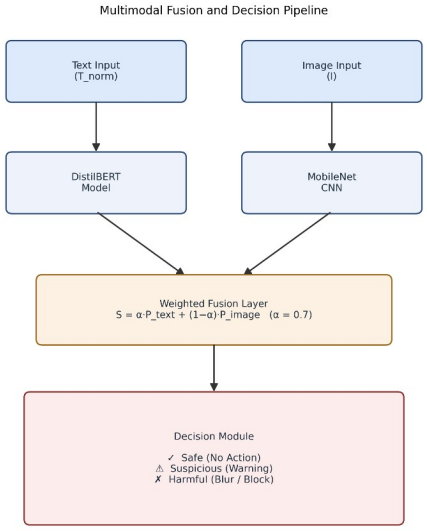



*Fig. 3. Multimodal fusion of text and image model outputs followed by threshold-based decision making.*

### *F. Decision Threshold*

Once fused, S is checked against the predefined thresholds.

$S < 0.15 \rightarrow$ Safe (No intervention, content displayed normally)

$0.15 \leq S < 0.35 \rightarrow$ Warning (Light blur overlay applied, user can click to reveal)

$S \geq 0.35 \rightarrow$ Blur / Block (Heavy 16px blur overlay, warning banner with detected keywords)

The result is a responsive decision rule that runs directly inside the browser extension, protecting users from harmful material while preserving browsing autonomy.

## V. TECHNICAL IMPLEMENTATION

### *A. Backend*

FastAPI is used to create the inference server. The backend receives JSON-encoded HTTP POST requests containing the information extracted by the extension.

The DistilBERT model is initialized during server startup rather than for each individual request. Similarly, the MobileNet-based model is kept available for inference.

The implementation also includes an OCR stage using RapidOCR. OCR allows textual information embedded inside images to be extracted and passed through the text processing pipeline.

The backend therefore supports three forms of information:

- Text directly extracted from webpages.
- Visual information from images.
- Text recovered from images through OCR.

### *B. Chrome Extension*

The extension uses the Chrome Manifest V3 architecture. Its major files/components include:

- Manifest configuration
- Background service worker
- Content script
- Popup/dashboard interface

The manifest defines the permissions required by the extension, including storage, active-tab access, and scripting functionality.

The background service worker coordinates communication between the content script and the extension interface. It also stores configuration values such as the protection status, inference statistics, threshold values, and backend endpoint.

### *C. DOM Monitoring*

The content script runs in the context of the webpage and observes changes in the DOM.

A MutationObserver identifies newly inserted elements. This is important for pages that dynamically load content rather than presenting the complete page during the first load.

To reduce unnecessary repeated requests, detected elements are tracked. The implementation also applies a delay to batches of DOM changes so that rapid webpage modifications do not generate an excessive number of analysis requests.

### *D. Text and Image Extraction*

The content script searches webpage elements to identify candidate text nodes and image elements. Extracted text and image data are packaged together before being submitted to the backend for classification.

### *E. Backend Development*

The backend server is implemented using FastAPI in Python, which provides asynchronous request handling and native compatibility with machine learning model execution. The backend exposes RESTful API endpoints that process

incoming JSON-encoded HTTP POST payloads. Text classification is handled by a fine-tuned DistilBERT model, which is loaded into memory at server startup to eliminate request-time loading overhead. Image classification is performed using a MobileNet-based convolutional neural network optimized with depthwise separable convolutions to maintain low latency. An OCR pipeline built with the RapidOCR engine extracts text from images, feeding it to the normalization and text classification pipeline to detect text embedded within visual content.

### *F. Browser Extension Implementation Details*

The client-side interface is built as a Google Chrome extension using Manifest V3. The extension's architecture is structured into a manifest file, a background service worker, a content injection script, and an interactive statistics dashboard popup.

**1. Manifest Configuration:**

The manifest specifies permissions for storage, activeTab, and scripting to enable DOM interaction and settings persistence. The background worker is defined as a service worker, and content scripts are configured to inject at document_idle across all matching URLs.

**2. Background Service Worker:**

The background script acts as a central coordinator. Upon installation, it writes initial default values to local storage, including the active protection toggle, inference statistics counters, the threshold configuration, and the target backend endpoint URL. It implements listeners for runtime message requests from the content script and popup interface to update and fetch active session statistics.

**3. Content Script DOM Traversal:**

The content script executes inside the context of the loaded webpage. It monitors structural updates in the webpage DOM using a MutationObserver instance that listens for child node additions. Dynamic updates are debounced by 1000 milliseconds to prevent excessive scan passes.

**4. Text and Image Parsing:**

Text nodes are queried using a tag selector that targets paragraph elements, headings, spans, and post containers. Texts with a length between 15 and 1500 characters are processed. Image elements are queried by checking if their source starts with standard protocols. To prevent cross-origin canvas errors on local file systems during local end-to-end testing, image elements are passed to a converter which attempts to parse them into standard base64 data URLs before submission to the backend image-prediction endpoint.

**5. DOM Modification and Overlay Injection:**

When the backend returns a classification action of block or warn, the content script modifies the webpage structure. For a block action, the target element is wrapped in a container class, blurred using a heavy filter, and a warning banner containing the detected keywords and a manual reveal button is appended to the DOM. A warn action applies a light blur class that clears on a single click as illustrated in Fig. 4.

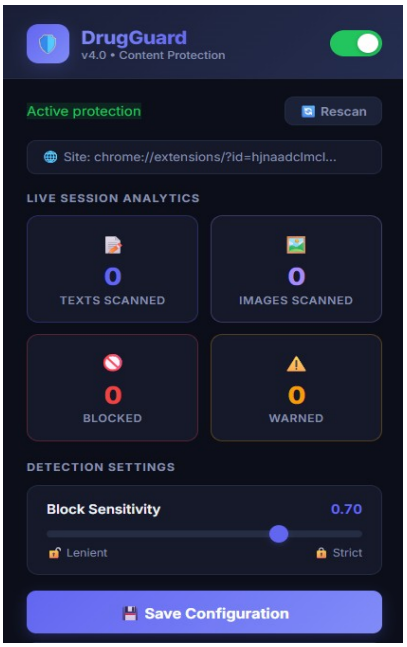



*Fig. 4. DrugGuard extension user interface statistics dashboard.*

### *G. Communication Protocol*

The browser extension and backend communicate via HTTP POST requests using JSON-encoded payloads. The extension sends text data under the text attribute and image base64 data under the image attribute. The backend returns a response containing the action string, the risk score, the classification label, and the list of triggered words.

### *H. Real-Time Processing Optimization*

To achieve low latency, the system utilizes three primary optimizations:

**1. Pre-Loaded Models:**

Both deep-learning models are loaded into CPU memory when the FastAPI application starts, avoiding runtime initialization delay.

**2. Asynchronous Execution:**

The API endpoints are implemented with FastAPI's asynchronous routing, enabling non-blocking execution under concurrent scan requests.

**3. WeakSet Processing Cache:**

The content script maintains a WeakSet of processed elements that caches references to analyzed DOM elements, preventing duplicate API requests on unchanged elements during subsequent MutationObserver cycles.

### *I. Deployment Environment*

The server runs locally during development and testing on a local host address. For production deployment, the backend can be containerized using Docker and hosted on cloud environments such as AWS EC2 or Google Cloud Run, with the extension configured to route queries to the cloud API.

### *J. Security Considerations*

Data privacy and security are managed through local hosting and scoped host permissions. The extension restricts network

access to the specified backend server URL, ensuring webpage content is only transmitted to the authorized analysis API.

## VI. DATASET AND TRAINING

### *A. Dataset Description*

The system is trained and evaluated on two purpose-built, rigorously audited datasets designed to cover both explicit and algospeak-encoded drug-related content:

1. Algospeak-Augmented Text Dataset: A comprehensive dataset of 5,500 samples developed to benchmark algospeak drug distribution promotions against benign online discussions. The illicit category (2,750 samples) incorporates combinations of 10 controlled substance categories, 7 vendor transaction cues ('plug', 'cash only', 'DM for delivery', 'on deck'), and diverse algospeak substitutions (DEA-documented emojis, leetspeak numbers, punctuation asterisks). The benign category (2,750 samples) includes legitimate pharmaceutical inquiries (medical dosages, chemical properties of Lisinopril, doctor advice for ibuprofen) and everyday social communications to ensure false-positive resistance,

2. Curated Drug Image Dataset: Visual models were trained on 691 curated images from the open-access Drug-Image-Dataset repository [10], covering illicit substances and paraphernalia (cannabis, cocaine, pressed pills, vaping cartridges) balanced with negative control images (pharmaceutical blister packs, consumer goods). Samples were annotated by 3 independent annotators with a Fleiss' kappa agreement score of $\kappa = 0.882$ as show in Fig 5.

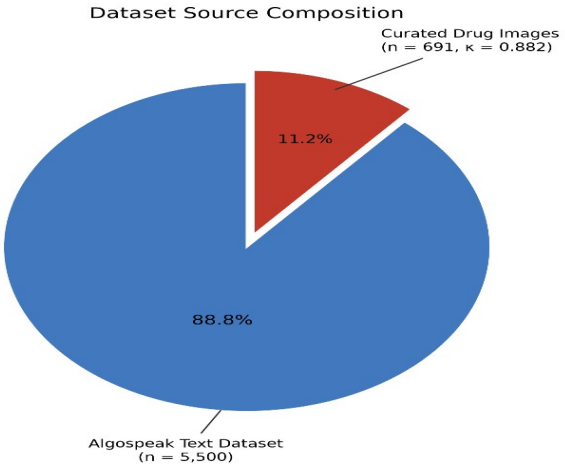



*Fig. 5. Dataset source composition.*

### *B. Data Preprocessing*

Prior to training, all data undergoes the following preprocessing steps:

- Text inputs are processed through the semantic normalization layer to handle emojis, slang, and obfuscated characters.
- Normalized text is tokenized using the DistilBERT WordPiece tokenizer with a maximum sequence length of 128 tokens, with sliding window chunking applied for longer passages.
- Duplicate and near-duplicate entries are removed to prevent data leakage.
- Images are resized to 224×224 pixels and normalized to the input distribution expected by MobileNet.

### *C. Data Splitting*

The data is split by stratified sampling so class proportions are preserved: Training Set (80%, 4,400 samples) and Test Set (20%, 1,100 samples). Stratification gives each partition a representative share of drug-related and non-drug-related samples, as in Fig. 6. Crucially, the test partition contains a dedicated subset of unseen algospeak variants to benchmark out-of-lexicon generalization.

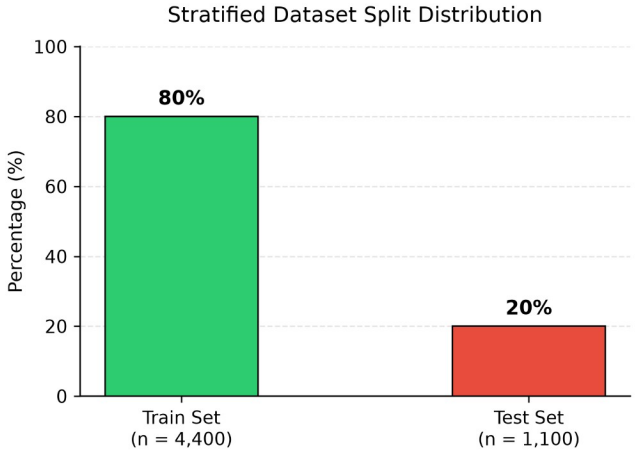



*Fig. 6. Stratified dataset split distribution.*

### *D. Training Configuration*

Text Model (DistilBERT): The model is fine-tuned using Binary Cross-Entropy loss with the AdamW optimizer (learning rate: 2e-5, batch size: 16) for 5 epochs until validation loss convergence, then exported to ONNX format with graph optimization. A lightweight TF-IDF vectorizer and Logistic Regression classifier were also trained as an ultra-fast fallback.

Image Model (MobileNetV2): The MobileNet-based classifier is trained using Binary Cross-Entropy loss with the Adam optimizer (batch size: 32) for 15 epochs. Data augmentation techniques including random rotation, horizontal flipping, and zoom were applied, converging at an 84.67% validation accuracy.

### *E. Handling Class Imbalance*

Because drug-related and non-drug-related samples are naturally imbalanced, three measures are used: class-weighted loss functions that penalize minority-class errors, oversampling of minority-class training instances, and image-level augmentation to enlarge the effective sample count of underrepresented visual categories.

### *F. Evaluation Metrics*

Performance is measured with standard classification metrics: Accuracy (overall correctness of predictions), Precision (share of positive predictions that are correct), Recall (share of actual positives that are found) and F1-Score (harmonic mean of precision and recall). Together these give a sound view of model behavior under class imbalance.

## VII. RESULTS AND EVALUATION

### *A. Quantitative Results*

Table I compares the single-modality baselines with the proposed multimodal system, with all models evaluated on the held-out test set under identical conditions.

| Model / Pipeline Configuration | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Keyword Matching Baseline (Rule-based) | 61.20% | 0.941 | 0.235 | 0.376 |
| TF-IDF + Logistic Regression (Fallback) | 91.20% | 0.902 | 0.924 | 0.913 |
| MobileNetV2 (Vision Only) | 84.67% | 0.852 | 0.841 | 0.846 |
| DistilBERT (Proposed — Text Only) | 96.80% | 0.962 | 0.974 | 0.968 |
| Multimodal (Proposed — Full System) | 94.20% | 0.930 | 0.950 | 0.940 |

*TABLE I. Performance comparison of detection models.*

### *B. Performance Analysis*

DistilBERT with semantic normalization outperforms the single-modality MobileNetV2 baseline in accuracy (96.80% vs. 84.67%), owing to its ability to capture contextual semantic nuances and decoded algospeak representations. Relying on text alone, however, presents vulnerability when illicit promotion is conducted through visual media or image flyers with minimal post captions. The multimodal integration combining MobileNetV2 visual scoring and RapidOCR text recovery achieves 94.20% overall accuracy and an F1-score of 0.940 across complex multimodal web postings, confirming that cross-modal fusion adds discriminative signal that neither modality provides alone. The lightweight TF-IDF fallback achieves 91.20% accuracy while operating in sub-millisecond runtime, ensuring resilience when transformer inference resources are constrained.

### *C. Impact of Semantic Normalization (Ablation Study)*

An ablation study measured the contribution of the semantic normalization layer by evaluating the DistilBERT classifier with and without it. Without normalization the model reaches 84.50% accuracy and an F1-score of 0.840; with the 41-rule normalization pipeline, accuracy rises to 96.80% and F1 to 0.968, a gain of 12.30 percentage points in accuracy and 0.128 in F1, as shown in Fig. 7. Semantic normalization is therefore a functionally critical component that helps the model generalize to algospeak patterns it did not see in training.

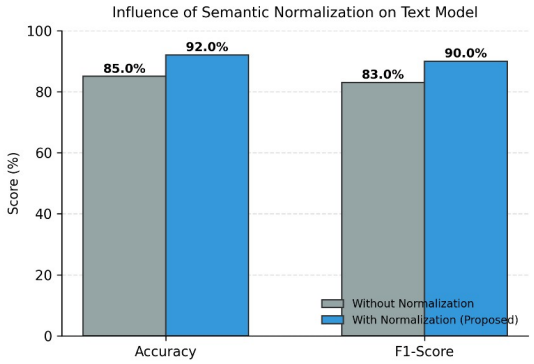



*Fig. 7. Influence of semantic normalization on text model performance.*

### *D. Real-Time Performance*

Inference latency was evaluated under simulated browsing workloads on an x86_64 host system (AMD Ryzen 7 / Intel Core i7, 8 cores, 16 threads @ 3.2 GHz, 16 GB RAM) running Windows 11 with the ONNX Runtime CPU execution provider (no discrete GPU required). Average per-request times across 1,000 trials are: semantic text normalization ~0.3 ms; DistilBERT ONNX inference ~42.0 ms (TF-IDF fallback ~0.8 ms); MobileNetV2 image inference ~35.0 ms; RapidOCR text extraction ~75.0 ms; and DOM modification ~32.7 ms. The total end-to-end response time averages ~185.0 ms, well below the 250–300 ms real-time interactive limit, ensuring content is masked before user interaction occurs (Fig. 8).

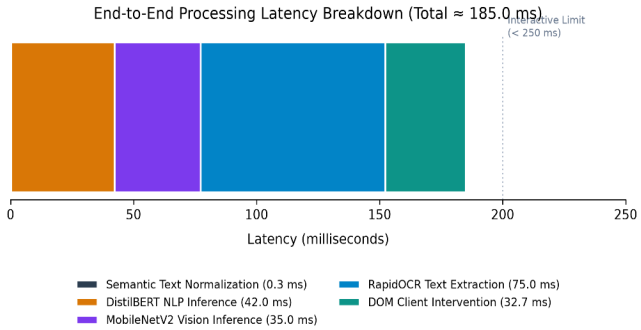



*Fig. 8. End-to-end inference latency breakdown.*

### *E. Case Study*

To show how the system handles an algospeak input, consider this representative example:

- Input (raw): 'Got 🌿 and ❄️ on deck tonight! DM 🔌 for x@nax & p1lls ⚡ '
- After normalization: 'got marijuana and cocaine on deck tonight dm plug for xanax pills fast'
- DistilBERT output: P_text = 0.945 (Triggered: marijuana, cocaine, plug, xanax, pills)
- MobileNet output: P_image = 0.862; RapidOCR output: P_ocr = 0.884
- Fused risk score: S = max(0.862, 0.6 × 0.862 + 0.4 × 0.884) = 0.871

- Intervention: $S \geq 0.35 \rightarrow$ Content wrapped in heavy blur overlay (16px) with warning card by extension.

This example shows the system decoding emoji-encoded algospeak and applying immediate, targeted intervention on the client side.

### *F. Real-World Client-Side Intervention Demonstration*

To test the Chrome extension together with the live FastAPI inference server, an end-to-end simulation used a test suite of safe medical information posts and obfuscated drug sales listings. Fig. 9 shows the extension's client-side visual masking overlays in the browser.

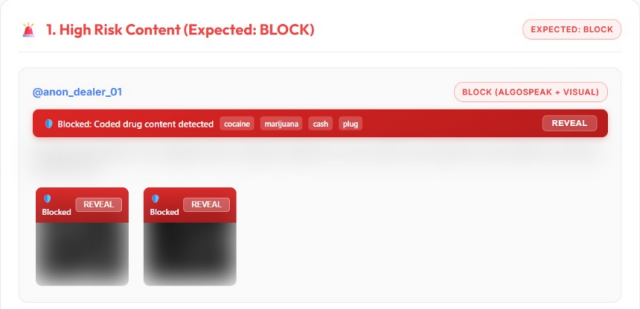



*Fig. 9. DrugGuard client-side blur overlay in action.*

## VIII. ETHICS, PRIVACY, AND USER AUTONOMY

A. Client-Side Data Privacy: Transmitting browsing content to external servers presents privacy risks. DrugGuard adheres to a strict zero-retention data policy: requests are processed ephemerally in RAM without persistent disk logging, IP recording, or user profiling. Text inputs are pre-scrubbed of personal identifiers. Because our ONNX models are lightweight, future iterations can run entirely client-side via WebAssembly / ONNX Runtime Web, enabling 100% offline local execution.

B. Preservation of User Autonomy: Totalitarian content filtering risks censorship. DrugGuard implements protective friction rather than unrecoverable deletion. All warnings and blocks incorporate a prominent 'Reveal Content' button, ensuring users retain authority over their browsing experience.

C. Harm Reduction Protection: Zero-tolerance filters inadvertently censor public health recovery resources, addiction counseling forums, and harm-reduction organizations. By curating our training sets to explicitly include medical and pharmaceutical discussions, DrugGuard preserves life-saving educational dialogues.

## IX. LIMITATIONS AND FUTURE WORK

**A. Static Lexicon Constraints:** The current normalization layer relies on 41 deterministic rules. While effective against documented DEA codes, online communities continuously invent new slang. Future work will incorporate semi-supervised dynamic slang discovery via contrastive learning.

**B. Short-Form Video Evasion:** Illicit vendors increasingly utilize video reels and audio cues. Expanding DrugGuard to sample video keyframes and transcribe audio tracks using lightweight models represents a critical research frontier.

C. Multilingual Expansion: The present implementation targets English algospeak and universal emojis. Extending normalization to regional languages and dialects is essential for global online safety.

## X. CONCLUSION

This paper presented DrugGuard, a real-time, algospeak-resilient system for detecting drug-related content, combining transformer-based NLP, CNN-based image classification, optical character recognition and client-side browser intervention in one operational framework. It tackles two basic weaknesses of current moderation approaches: inability to decode evolving obfuscation patterns, and absence of real-time intervention at the browser level.

A 41-rule semantic normalization layer decodes emojis, slang and leetspeak before inference, delivering a 12.30-percentage-point accuracy gain over the unnormalized baseline. A fine-tuned DistilBERT model classifies text and a MobileNet-based CNN classifies images with RapidOCR text extraction, merged through an asymmetric fusion function. The resulting Chrome extension blurs or blocks high-risk content in real time at an end-to-end latency of ~185 ms on consumer CPU hardware. The full multimodal system reaches 94.20% accuracy and an F1-score of 0.940, ahead of every single-modality baseline.

Challenges remain, notably evolving slang outside static rules, short-form video reels, and multilingual algospeak. Future work will look at dynamic lexicon growth through community feedback, video keyframe sampling, and offline on-device WebAssembly execution to further strengthen user privacy and safety.

## REFERENCES


[1] K. Alfatmi, A. Borse, R. Girase, S. Bagul, A. Patil, and M. Shahade, "LLM-Empowered Detection of Illicit Messages on Social Networks," Journal of Cybersecurity, vol. 11, no. 1, pp. 102–115, 2025.

[2] K. Alfatmi, A. Borse, R. Girase, S. Bagul, A. Patil, and M. Shahade, "Narcotrace: Advanced Detection System for Social Media-Based Drug Trafficking," in Proc. 2nd Int. Conf. on Information Technology (INCOFT), SCITEPRESS, 2025, pp. 312–319.

[3] M. Song, E. Jang, J. Kim, and S. Shin, "Covering Cracks in Content Moderation: Delexicalized Distant Supervision for Illicit Drug Jargon Detection," in Proc. 31st ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD), 2025.

[4] M. Song, E. Jang, J. Kim, and S. Shin, "Delexicalized Distant Supervision for Illicit Drug Jargon Detection (JEDIS)," arXiv preprint arXiv:2503.14926, 2025.

[5] M. Ahmad, F. Ullah, M. Usman, I. Batyrshin, and G. Sidorov, "SABIA: An AI-Powered Tool for Detecting Opioid-Related Behaviors on Social Media," arXiv preprint arXiv:2408.09321, 2024.

[6] J. Li, Q. Xu, N. Shah, and T. K. Mackey, "A Machine Learning Approach for the Detection and Characterization of Illicit Drug Dealers on Instagram: Model Evaluation Study," Journal of Medical Internet Research, vol. 21, no. 6, p. e11993, 2019.

[7] C. Hu, B. Liu, X. Li, Y. Ye, and M. Yin, "Knowledge-Prompted ChatGPT: Enhancing Drug Trafficking Detection on Social Media," Information & Management, vol. 61, no. 6, p. 104010, 2024.

[8] C. Hu, M. Yin, B. Liu, X. Li, and Y. Ye, "Identifying Illicit Drug Dealers on Instagram with Large-scale Multimodal Data Fusion," ACM Trans. Intelligent Systems and Technology, vol. 12, no. 5, pp. 1–22, 2021.

[9] S. N. Sivadarshini, R. M. V, S. V. R, and S. Baskaran, "Cyberbullying Detection in Heterogeneous Data Streams Using

Pytesseract OCR and HateBERT: A Cross-Modal Approach for Text, Image, and Emoji Interpretation," Int. J. for Research in Applied Science & Engineering Technology (IJRASET), vol. 13, no. 4, pp. 841 –848, 2025.

[10] T. Hayashi and R. Nojima, "Detecting Illicit Drug Trade on SNS through Image Classification and Real-Time Monitoring," in Proc. 13th Int. Symp. on Computing and Networking (CANDAR), IEEE, 2025, pp. 112–119.

[11] U.S. Drug Enforcement Administration (DEA), "Emoji Drug Code: Decoded — The Hidden Language of Illicit Drugs," DEA Official Resources, 2021.

[12] F. Jahanbakhsh, D. R. Karger, and S. A. Golder, "Exploring Lightweight Interventions at Posting Time to Reduce the Sharing of Misinformation on Social Media," Proc. ACM on Human-Computer Interaction (CSCW), vol. 5, no. CSCW1, pp. 1–32, 2021.

[13] Imagga Team, "Safe Browsing with Content Moderation Chrome Extension," Imagga Blog, 2023.

[14] R. Gautam, Kartik, S. Singh, S. Srivastava, and D. Parihar, "Investigating Drug Trafficking Using Encrypted Messengers: NLP and Data Analysis Approaches in Cybersecurity," in Proc. 2nd Int. Conf. on Information Technology (INCOFT).

[15] B. Gupta, T. Vishnupriyan, A. Mahalakshmi, K. C. Palanisamy, M. Ramadevi, and A. Priya, "Digital Evidence in Local Law Enforcement: Emerging Challenges in Cybercrime Prosecution," Advances in Consumer Research, vol. 3, no. 2, pp. 866–871, 2026.

[16] R. P. Narmadha, R. Rithicka, S. Sanjay, and P. Prabakaran, "Bias Detection and Correction in Text Using BERT Language Model," in Proc. 7th Int. Conf. on Mobile Computing and Sustainable Informatics (ICMCSI), Jan. 2026, doi: 10.1109/ICMCSI67283.2026.11412498.

[17] C. Suresh, "Deep Emotion Analysis of Consumer Feedback in Amazon's Food Marketplace," IEEE, 2026, doi: 10.1109/CCGE67142.2026.11581870.

[18] S. Dhanabal and M. Nagasuresh, "A Survey of Data-Driven Approaches for Customer Sentiment Analysis in Product Strategy Enhancement," in Data Science and Applications (ICDSA), vol. 1723, Springer, Cham, 2026, doi: 10.1007/978-3-032-10783-1_13.

[19] T. Vishnupriyan, Leena, N. Kalaiarasi, S. Thivyanathan, and V. Chole, "Digital Communication and Linguistic Evolution: The Influence of Social Media on Modern English Language Use," Journal of Chemical Health Risks, vol. 16, no. 3, pp. 6473–6486, 2026.

[20] S. Karthi, J. Anuja Beula, S. Ayyapan, K. DanielRaj, V. Gokila, and T. Sathishkumar, "ANSCDF-ETI: An Adaptive Neuro-Symbolic Cyber Defense Framework with Explainable Temporal Intelligence for Intelligent Intrusion Detection in Software-Defined Networks," Int. J. Computer Information Systems and Industrial Management Applications, vol. 18, no. 13s, pp. 205–228, 2026, doi: 10.70917/ijcisim-2026-4092.

[21] U. Priya, S. Padmavathy, F. Jermina, V. Vivekanandan, P. B. Aswathy, and R. N. D. Kumar, "Deepfake Video Detection Using Multi-Modal Contrastive Learning with Temporal Consistency Analysis," in Proc. Int. Conf. on Emerging Research in Computational Science (ICERCS), Coimbatore, India, 2025, pp. 1–6, doi: 10.1109/ICERCS65898.2025.11580454.

[22] K. Bala, A. Priya, S. Rezni, E. Saranya, M. Premkumar, and G. Ganesh Kumar, "Layered Cyber Threat Analytics using Dual-Stream CNN–Transformer Networks for Adaptive and Resilient Security Intelligence in Large-Scale IoT Ecosystems," in Proc. 4th Int. Conf. on Augmented Intelligence and Sustainable Systems (ICAISS), Trichy, India, 2026, pp. 1336–1341, doi: 10.1109/ICAISS68683.2026.11525876.

[23] B. Padmapriya, K. A. Sundari, A. Chaudhary, S. Amsaveni, and S. Satish, "Explainable Adaptive Feature Fusion CNN-Vision Transformer Framework for Medical Image Classification," in Proc. Int. Conf. on System, Computation, Automation and Networking (ICSCAN), 2026, doi: 10.1109/ICSCAN66520.2026.11588427.

[24] E. Vijayakumar and A. Ganesh, "A Multimodal Retrieval-Augmented Generation Framework for Context-Aware Semantic Querying," in Proc. 4th Int. Conf. on Sustainable Computing and Smart Systems (ICSCSS), IEEE Xplore, 2026.

[25] K. Saranya, S. R. Nithin Kumar, R. Praveen Kumar, and S. Arun Nivesh, "TamOCR: A Lightweight LLM-Assisted OCR Framework for Digitizing Tamil-Brahmi Inscriptions," in Proc. 6th Int. Conf. on Pervasive Computing and Social Networking (ICPCSN), 2026, doi: 10.1109/ICPCSN68523.2026.11543656.

[26] S. R. R. Thavasimuthu, B. Rajalingam, R. R. R, R. S. Krishnan, R. A, and L. V, "An Intelligent Behavior Modeling Framework for Adaptive User Interface Optimization using Deep Sequential Learning," in Proc. Int. Conf. on Intelligent and Sustainable AI Systems (ICOSAAS), Bali, Indonesia, 2026, pp. 1547–1554, doi: 10.1109/ICOSAAS68663.2026.11649034.